CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: PetFriend

Prepared By: Canonigo, Johndaniel Agum

Version: 2.0

Date: 05/27/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/26	Canonigo, Johndaniel A.	Initial draft	Final
1.0	02/28/26	Canonigo, Johndaniel A.	4.0 - 8.0	Final
2.0	05/27/26	Canonigo, Johndaniel A.	4.0 and 5.0	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

PetFriend is a peer-to-peer pet care platform that connects pet owners with verified pet sitters within the university community and surrounding areas. The system enables users to book pet sitting services, including dog walking, feeding, overnight care, and playtime sessions, through a streamlined web and mobile application. The platform ensures secure transactions, user verification, and reliable service delivery for both pet owners and student sitters seeking flexible income.

1.2 Objectives

1. Develop a fully functional pet sitting marketplace with user authentication, service booking, and profile management
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for seamless communication between all system components
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Deploy all system components to production-ready environments

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Dual user roles: Pet Owner and Pet Sitter
- Pet profile creation and management
- Sitter profile with availability, rates, and experience
- Service booking with date/time selection
- Sandbox payment processing integration (testing environment only – no real money transactions)
- Rating and review system for sitters

- Messaging/chat between owners and sitters
- Admin panel for user verification and dispute resolution
- Responsive web interface
- Native Android mobile application
- PostgreSQL database with relational schema

Excluded Features:

- Real-time GPS tracking of pet walks
- Video calling integration
- Insurance coverage for pets
- Automated background checks (manual verification only)
- Multi-language support
- Push notifications (basic email/SMS only)

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the **PetFriend** system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly. The intended audience includes:

- **Stakeholders** (product owners, project managers) to validate system scope and functionality
- **Developers** (frontend/backend engineers) to implement features based on defined requirements
- **QA Testers** to design test cases aligned with functional and non-functional criteria
- **System Architects** to review integration points and technology stack decisions

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: PetFriend
Domain: Pet Care Services / Peer-to-Peer Marketplace
Primary Users: Pet Owners, Pet Sitters, Administrators
Problem Statement: Pet owners (especially students and faculty) struggle to find reliable, affordable pet care during exams, breaks, travel, or busy schedules. Simultaneously, responsible students seek flexible side income opportunities. Existing platforms lack university-specific verification and community trust.

Solution: A localized pet sitting marketplace focused on the university ecosystem, featuring verified student sitters, transparent pricing, secure booking, and mutual rating systems to build trust and reliability.

2.2 Core User Journeys

Journey 1: First-time Pet Owner Booking

1. User visits PetFriend web application
2. Clicks "Sign Up" and creates account as Pet Owner
3. Completes profile and adds pet details (name, breed, age, special needs)
4. Browses available pet sitters by location and availability
5. Views sitter profile (ratings, experience, hourly rate)
6. Selects service type (walk, feeding, overnight) and preferred date/time
7. Confirms booking and processes payment
8. Receives booking confirmation with sitter contact details
9. After service completion, rates and reviews the sitter

Journey 2: Student Becoming a Pet Sitter

1. User registers account as Pet Sitter
2. Completes sitter profile (availability, rates, experience, certifications)
3. Uploads verification documents (student ID, references)
4. Waits for admin approval
5. Once approved, appears in search results for pet owners

6. Receives booking requests and accepts/rejects sessions
7. Completes service and receives payment
8. Builds reputation through owner ratings

Journey 3: Administrator User Verification

1. Admin logs in with admin credentials
2. Navigates to pending sitter applications
3. Reviews submitted documents and background information
4. Approves or rejects sitter applications
5. Monitors user reports and resolves disputes
6. Views platform analytics (bookings, revenue, active users)

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Dual role selection (Pet Owner / Pet Sitter)
3. Pet profile management (add, edit, delete pets)
4. Sitter profile management (availability, rates, bio)
5. Service booking with calendar integration
6. Sandbox payment simulation for bookings (no real financial transactions)
7. Rating and review system
8. Admin panel for user verification

SHOULD HAVE

1. In-app messaging between owners and sitters
2. Booking history and upcoming sessions dashboard
3. Email notifications for booking confirmations
4. Search and filter sitters by location, availability, rating
5. Responsive design for all screen sizes

COULD HAVE

1. Pet care tips and resources section
2. Loyalty program for frequent users
3. Group booking for multiple pets
4. Emergency contact integration

WON'T HAVE

1. Real-time GPS tracking
2. Video calling features
3. Insurance integration
4. Automated background checks
5. Multi-language support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password, Full Name, Role Selection
- **Validation:** Email format, password strength (≥8 characters), role selection required
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Pet Profile Management

- **Screens:** Add Pet, Pet List, Edit Pet
- **Fields:** Pet Name, Breed, Age, Weight, Special Needs, Vaccination Status, Photo Upload
- **Functions:** Add pet, edit pet details, delete pet, view pet list
- **API Endpoints:** GET /pets, POST /pets, PUT /pets/{id}, DELETE /pets/{id}
- **Validation:** Required fields, photo size limits (max 5MB)

Feature: Sitter Profile Management

- **Screens:** Sitter Profile Setup, Availability Calendar, Rate Settings
- **Fields:** Bio, Experience, Certifications, Hourly Rate, Availability Schedule, Profile Photo
- **Functions:** Set availability, update rates, upload verification documents
- **API Endpoints:** GET /sitters/profile, PUT /sitters/profile, POST /sitters/availability
- **Admin Approval:** Pending → Approved/Rejected workflow

Feature: Service Booking

- **Screens:** Sitter Search, Booking Form, Confirmation Page
- **Service Types:** Dog Walk (30min/1hr), Feeding Visit, Overnight Stay, Playtime Session
- **Booking Flow:** Select sitter → Choose date/time → Select service → Confirm payment → Receive confirmation
- **API Endpoints:** POST /bookings, GET /bookings/{id}, PUT /bookings/{id}/cancel
- **Calendar Integration:** Real-time availability checking

Feature: Payment Processing (Sandbox Mode Only)

- **Payment Methods:** Simulated Credit/Debit Card and E-Wallet (Test Mode Only)
- **Pricing Model:** Hourly rates set by sitters + platform service fee (10%)
- **Payment Flow:**
    - User enters test payment details
    - System validates format (no real transaction occurs)
    - Booking marked as "Paid (Sandbox)"
    - Payment status stored for simulation purposes only
- **API Endpoints:** POST /payments/process, GET /payments/{id}
- **Security:** No real card data stored; test tokens or dummy payment credentials only

Note: This system does NOT process real financial transactions. All payments are simulated for academic/testing purposes.

Feature: Rating & Review System

- **Screens:** Submit Review, View Reviews
- **Rating Scale:** 1-5 stars + written review
- **Fields:** Overall rating, communication, reliability, pet care quality
- **Functions:** Submit review after service completion, view sitter ratings
- **API Endpoints:** POST /reviews, GET /sitters/{id}/reviews
- **Validation:** One review per booking, minimum 50 characters for written review

Feature: Admin Panel

- **Screens:** User Management, Sitter Verification, Booking Oversight, Reports
- **Functions:** Approve/reject sitter applications, view all bookings, resolve disputes, generate reports
- **Access Control:** Admin role required
- **API Endpoints:** Admin-prefixed endpoints with role validation

2.5 Acceptance Criteria

AC-1: Successful User Registration

Given I am a new user

When I enter valid email and strong password

And select my role (Pet Owner or Pet Sitter)

And click "Create Account"

Then my account should be created

And I should be automatically logged in

And redirected to role-specific onboarding

AC-2: Pet Owner Books a Service

Given I am logged in as a Pet Owner

And I have at least one pet in my profile

When I search for available sitters

And select a sitter with open availability

And choose service type, date, and time

And confirm booking with payment

Then I should receive booking confirmation

And the sitter should be notified

And the time slot should be blocked for other bookings

AC-3: Sitter Completes Profile Setup

Given I am logged in as a Pet Sitter

When I fill out my profile (bio, experience, rates)

And upload verification documents

And set my availability calendar

And submit for approval

Then my profile status should be "Pending Approval"

And I should receive email notification when approved

And my profile should appear in search results for pet owners

AC-4: Admin Approves Sitter Application

Given I am logged in as an Administrator

When I view pending sitter applications

And review submitted documents

And click "Approve"

Then the sitter's status should update to "Approved"

And the sitter should receive approval notification

And the sitter should be able to receive bookings

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time must be ≤ 2 seconds for 95% of all requests
- Web page load time must be ≤ 5 seconds on broadband connections (≥25 Mbps)
- Mobile app cold start time must be ≤ 3 seconds on mid-range Android devices (API Level 32+)
- System must support at least 100 concurrent users without degradation
- Database queries must complete within 500 milliseconds
- Search results must return within 3-5 second for sitter/pet queries

3.2 Security Requirements

- All data transmission must use HTTPS/TLS 1.3 encryption
- Authentication must use JWT token-based system with 2-hour expiration
- Passwords must be hashed using bcrypt with 12 salt rounds before storage
- SQL injection prevention via parameterized queries/prepared statements
- XSS protection through input sanitization and output encoding
- Rate limiting enforced at 100 requests per minute per IP address

- Role-based access control (RBAC) for all admin endpoints
- Payment processing will use sandbox/test mode only (e.g., Stripe Test Mode) with no live financial transactions
- Session tokens must be invalidated immediately on logout
- Sensitive user data (passwords, payment info) must never be logged

3.3 Compatibility Requirements

- Web Browsers: Chrome, Firefox, Safari, Edge (latest 2 versions)
- Mobile Platforms: Android 14.0+ (API Level 32+) for native app
- Screen Sizes:
    - Mobile: 360px width minimum
    - Tablet: 768px width minimum
    - Desktop: 1024px width minimum
- Operating Systems:
    - Windows 10 or later
    - macOS 10.15 (Catalina) or later
    - Linux Ubuntu 20.04 LTS or later
- Database: PostgreSQL 14+ compatible

3.4 Usability Requirements

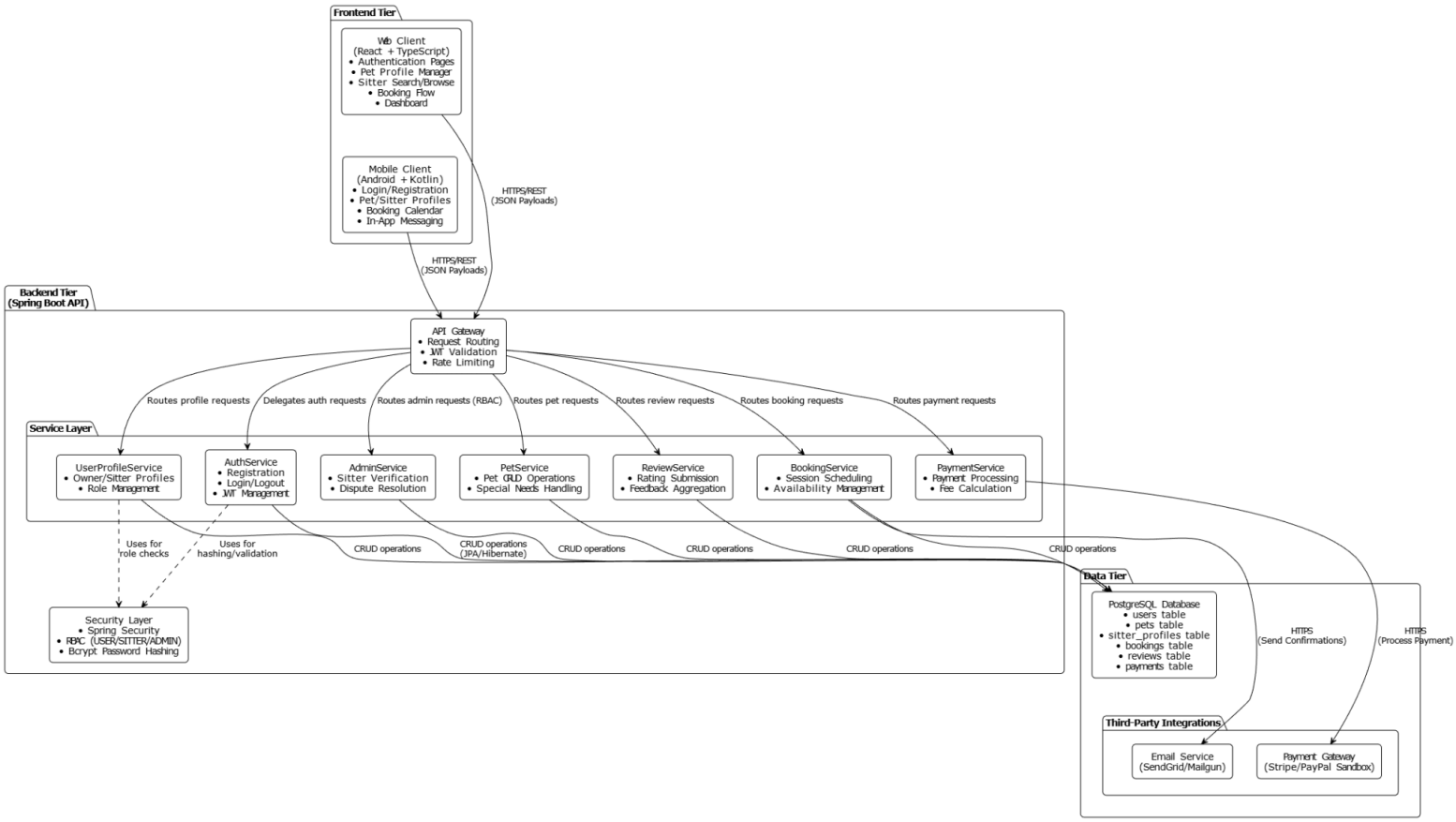
- First-time users must complete registration + first booking within 7 minutes
- Web interface must comply with WCAG 2.1 Level AA accessibility standards
- Navigation must remain consistent across all pages and platforms
- Error messages must be clear, actionable, and include recovery guidance (e.g., "Password must be 8+ characters with 1 number")
- Touch targets on mobile must be minimum 44×44 pixels
- Full keyboard navigation support for web interface (tab order, focus states)
- Real-time form validation with inline feedback (no submit-only validation)
- Loading states must display spinners/indicators for all async operations (>500ms)

- Critical actions (booking, payment) must include confirmation dialogs to prevent errors

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack

Backend

- Java 17
- Spring Boot 3.5.x (the repository uses Spring Boot 3.5.11)
- Spring Security with JWT for authentication and role-based access control
- Spring Data JPA / Hibernate (ORM)
- RestTemplate / WebClient for PayMongo and Supabase HTTP calls

Database

- PostgreSQL 14+

Web Frontend

- Next.js (App Router) 16.1.6
- React 19
- Native Next/Fetch for API calls
- Tailwind CSS and custom styles

Mobile

- Android (Kotlin)
- Retrofit + OkHttp for HTTP calls
- Coroutines for asynchronous work
- Google Sign-In (requestIdToken) integrated with Supabase exchange

Infrastructure & Hosting

- Mobile: APK distribution

Third-Party Integrations (actual project choices)

- PayMongo (sandbox) — checkout session creation + webhook-based payment status updates. Backend stores `paymentId` and updates booking/payment records on webhook events.
- Supabase Auth — used as a Google OAuth token-exchange helper for mobile Google Sign-In flows. The backend validates or exchanges Supabase-provided tokens and issues application JWTs.
- SMTP — used to send welcome emails from backend.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL: https://api.petfriend.app/api/v1

Protocol: HTTPS (TLS 1.3)

Format: JSON for all requests and responses

Authentication: Bearer token (JWT) in Authorization header Format: Authorization: Bearer {token}

Rate Limiting: 100 requests per minute per user/IP

Timestamp Format: ISO 8601 (2026-02-28T14:30:00Z)

Pagination: Query parameters: ?page=1&limit=20

Standard Response Structure:

```
{
  "success": true,
  "data": { /* resource payload */ },
  "error": null,
  "timestamp": "2026-02-28T14:30:00Z"
}
```

Error Response Structure:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2026-02-28T14:30:00Z"
}
```

5.2 Endpoint Specifications (key endpoints used in project)

All endpoints in the running application are prefixed with /api.

Authentication

- POST /api/auth/register — register with email/password (returns application JWT)
- POST /api/auth/login — login with email/password (returns application JWT)
- POST /api/auth/logout — revoke refresh token / logout
- POST /api/auth/refresh — refresh access token
- POST /api/auth/google — backend endpoint used after Supabase Google token exchange; accepts a Supabase access token or id_token, validates/exchanges it and returns application JWT

Uploads / Media

- POST /api/uploads/users/{userId}/photo — upload user profile photo (multipart/form-data)
- POST /api/uploads/pets/{petId}/photo — upload pet photo

Pets

- GET /api/pets
- POST /api/pets
- GET /api/pets/{petId}
- PUT /api/pets/{petId}
- DELETE /api/pets/{petId}

Sitters

- GET /api/sitters/profile
- PUT /api/sitters/profile
- POST /api/sitters/profile/verify
- GET /api/sitters/search

Bookings

- POST /api/bookings — creates booking and returns booking id + pricing
- GET /api/bookings
- GET /api/bookings/{bookingId}
- POST /api/bookings/{bookingId}/cancel
- POST /api/bookings/{bookingId}/complete

Payments (PayMongo integration)

- POST /api/payments/paymongo/checkout — create PayMongo checkout session. Returns { checkoutUrl, paymentId }.
- POST /api/payments/paymongo/webhook — PayMongo webhook listener. Backend validates webhook payload and updates booking/payment records.
- GET /api/payments/{paymentId}

Admin

- GET /api/admin/sitters/pending
- POST /api/admin/sitters/{sitterId}/approve
- POST /api/admin/sitters/{sitterId}/reject

5.3 Error Handling

HTTP Status Codes (standard usage)

- 200 OK: success
- 201 Created: resource created
- 400 Bad Request: validation errors or malformed input
- 401 Unauthorized: missing/invalid JWT
- 403 Forbidden: insufficient role/permissions
- 404 Not Found: resource not found
- 409 Conflict: duplicate resource
- 422 Unprocessable Entity: business rule violation (e.g., slot already booked)
- 429 Too Many Requests: rate limiting
- 500 Internal Server Error: unexpected failures

Error Object

All error responses use the error field in the standard envelope and include an application error code. Example:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "BUSINESS-001",
    "message": "Time slot unavailable",
    "details": "Sitter usr_54321 is already booked for 2026-03-01 09:00-10:00"
  },
  "timestamp": "2026-02-28T14:30:00Z"
```

```
}
```

Common application error codes used in the codebase include AUTH-001, VALID-001, DB-001, BUSINESS-001, PAYMENT-001, and SYSTEM-001.

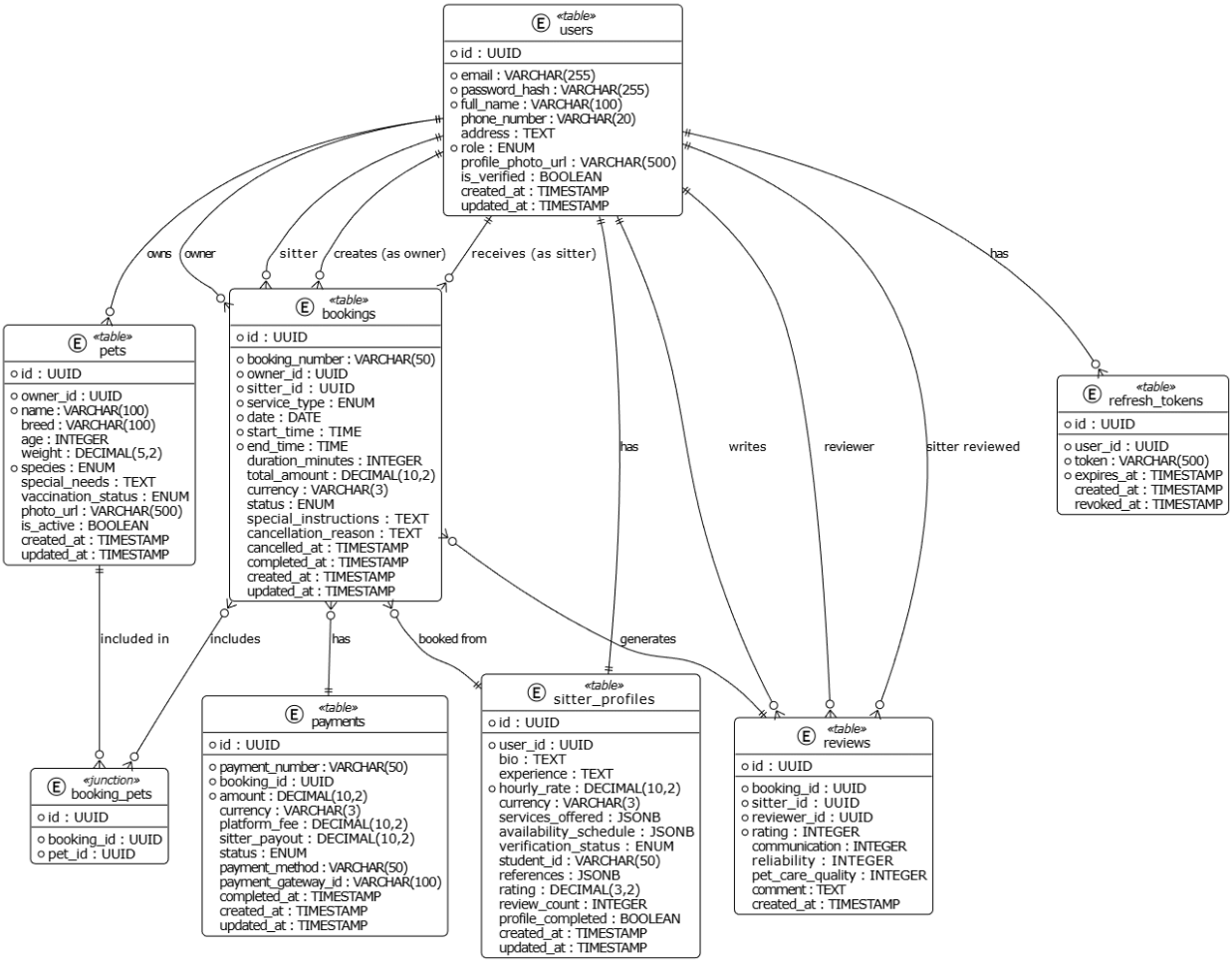
5.4 API Security Practices

- JWT: short-lived access token (2 hours) and refresh tokens (longer). Backend validates tokens on protected endpoints.
- Passwords: bcrypt hashing (no plaintext storage).
- Input validation: server-side validation on all POST/PUT endpoints.
- SQL injection: prevented via JPA parameterization.
- CORS: configured to allow the frontend origins defined via environment variables; webhook endpoints are explicitly permitted for PayMongo.
- Secrets: PayMongo secret key and Supabase service role key are stored in environment variables in production (PAYMONGO_SECRET_KEY, SUPABASE_SERVICE_ROLE_KEY, etc.).

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD



Detailed Relationships:

One-to-One:

- User ↔ Sitter Profile (Each user who is a sitter has exactly one sitter profile)

One-to-Many:

- User → Pets (One user can own multiple pets)
- User → Bookings (One user can create multiple bookings as owner)
- User → Reviews (One user can write multiple reviews)
- Sitter Profile → Bookings (One sitter can have multiple bookings)
- Booking → Reviews (One booking can generate one review)

Many-to-One:

- Booking → User (as owner) (Multiple bookings reference one pet owner)
- Booking → User (as sitter) (Multiple bookings reference one sitter)
- Booking → Pet (Multiple bookings can reference the same pet)
- Review → User (as reviewer) (Multiple reviews reference one reviewer)
- Review → User (as sitter) (Multiple reviews reference one sitter)
- Payment → Booking (Each payment belongs to one booking)

Many-to-Many:

- Booking ↔ Pet (One booking can include multiple pets, one pet can be in multiple bookings)

Key Tables:

1. users - User accounts and authentication
2. pets - Pet profiles and information
3. sitter_profiles - Pet sitter details and availability
4. bookings - Service booking records
5. booking_pets - Junction table for many-to-many relationship
6. reviews - Sitter ratings and feedback
7. payments - Payment transaction records
8. refresh_tokens - JWT refresh tokens for authentication

Table Structure Summary:

users:

- user_id (UUID, PRIMARY KEY)
- email (VARCHAR(255), UNIQUE, NOT NULL)
- password_hash (VARCHAR(255), NOT NULL)
- first_name (VARCHAR(100), NOT NULL)

- last_name (VARCHAR(100), NOT NULL)
- phone_number (VARCHAR(20))
- address (TEXT)
- role (ENUM: 'PET_OWNER', 'PET_SITTER', 'ADMIN', NOT NULL)
- profile_photo_url (VARCHAR(500))
- is_verified (BOOLEAN, DEFAULT false)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- last_login_at (TIMESTAMP, NULLABLE)

pets:

- id (UUID, PRIMARY KEY)
- owner_id (UUID, FOREIGN KEY → users.id, NOT NULL)
- name (VARCHAR(100), NOT NULL)
- breed (VARCHAR(100))
- age (INTEGER)
- weight (DECIMAL(5,2))
- species (ENUM: 'DOG', 'CAT', 'BIRD', 'OTHER', NOT NULL)
- special_needs (TEXT)
- vaccination_status (ENUM: 'UP_TO_DATE', 'OVERDUE', 'UNKNOWN', DEFAULT 'UNKNOWN')
- photo_url (VARCHAR(500))
- is_active (BOOLEAN, DEFAULT true)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

sitter_profiles:

- id (UUID, PRIMARY KEY)
- user_id (UUID, FOREIGN KEY → users.id, UNIQUE, NOT NULL)
- bio (TEXT)
- experience (TEXT)
- hourly_rate (DECIMAL(10,2), NOT NULL)
- currency (VARCHAR(3), DEFAULT 'PHP')
- services_offered (JSONB array: ['WALK', 'FEEDING', 'OVERNIGHT', 'PLAYTIME'])
- availability_schedule (JSONB – stores weekly availability)
- verification_status (ENUM: 'PENDING', 'APPROVED', 'REJECTED', DEFAULT 'PENDING')
- student_id (VARCHAR(50), NULLABLE)
- references (JSONB array – stores reference contacts)

- rating (DECIMAL(3,2), DEFAULT 0.00)
- review_count (INTEGER, DEFAULT 0)
- profile_completed (BOOLEAN, DEFAULT false)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

bookings:

- id (UUID, PRIMARY KEY)
- booking_number (VARCHAR(50), UNIQUE, NOT NULL)
- owner_id (UUID, FOREIGN KEY → users.id, NOT NULL)
- sitter_id (UUID, FOREIGN KEY → users.id, NOT NULL)
- service_type (ENUM: 'WALK', 'FEEDING', 'OVERNIGHT', 'PLAYTIME', NOT NULL)
- date (DATE, NOT NULL)
- start_time (TIME, NOT NULL)
- end_time (TIME, NOT NULL)
- duration_minutes (INTEGER, NOT NULL)
- total_amount (DECIMAL(10,2), NOT NULL)
- currency (VARCHAR(3), DEFAULT 'PHP')
- status (ENUM: 'PENDING', 'CONFIRMED', 'CANCELLED', 'COMPLETED', DEFAULT 'PENDING')
- special_instructions (TEXT)
- cancellation_reason (TEXT, NULLABLE)
- cancelled_at (TIMESTAMP, NULLABLE)
- completed_at (TIMESTAMP, NULLABLE)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

booking_pets:

- id (UUID, PRIMARY KEY)
- booking_id (UUID, FOREIGN KEY → bookings.id, NOT NULL)
- pet_id (UUID, FOREIGN KEY → pets.id, NOT NULL)
- UNIQUE (booking_id, pet_id)

reviews:

- id (UUID, PRIMARY KEY)
- booking_id (UUID, FOREIGN KEY → bookings.id, UNIQUE, NOT NULL)
- sitter_id (UUID, FOREIGN KEY → users.id, NOT NULL)
- reviewer_id (UUID, FOREIGN KEY → users.id, NOT NULL)

- rating (INTEGER, CHECK (rating >= 1 AND rating <= 5), NOT NULL)
- communication (INTEGER, CHECK (communication >= 1 AND communication <= 5))
- reliability (INTEGER, CHECK (reliability >= 1 AND reliability <= 5))
- pet_care_quality (INTEGER, CHECK (pet_care_quality >= 1 AND pet_care_quality <= 5))
- comment (TEXT)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

payments:

- id (UUID, PRIMARY KEY)
- payment_number (VARCHAR(50), UNIQUE, NOT NULL)
- booking_id (UUID, FOREIGN KEY → bookings.id, UNIQUE, NOT NULL)
- amount (DECIMAL(10,2), NOT NULL)
- currency (VARCHAR(3), DEFAULT 'PHP')
- platform_fee (DECIMAL(10,2), NOT NULL)
- sitter_payout (DECIMAL(10,2), NOT NULL)
- status (ENUM: 'PENDING', 'COMPLETED', 'FAILED', 'REFUNDED', DEFAULT 'PENDING')
- payment_method (VARCHAR(50))
- payment_gateway_id (VARCHAR(100), NULLABLE)
- completed_at (TIMESTAMP, NULLABLE)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

refresh_tokens:

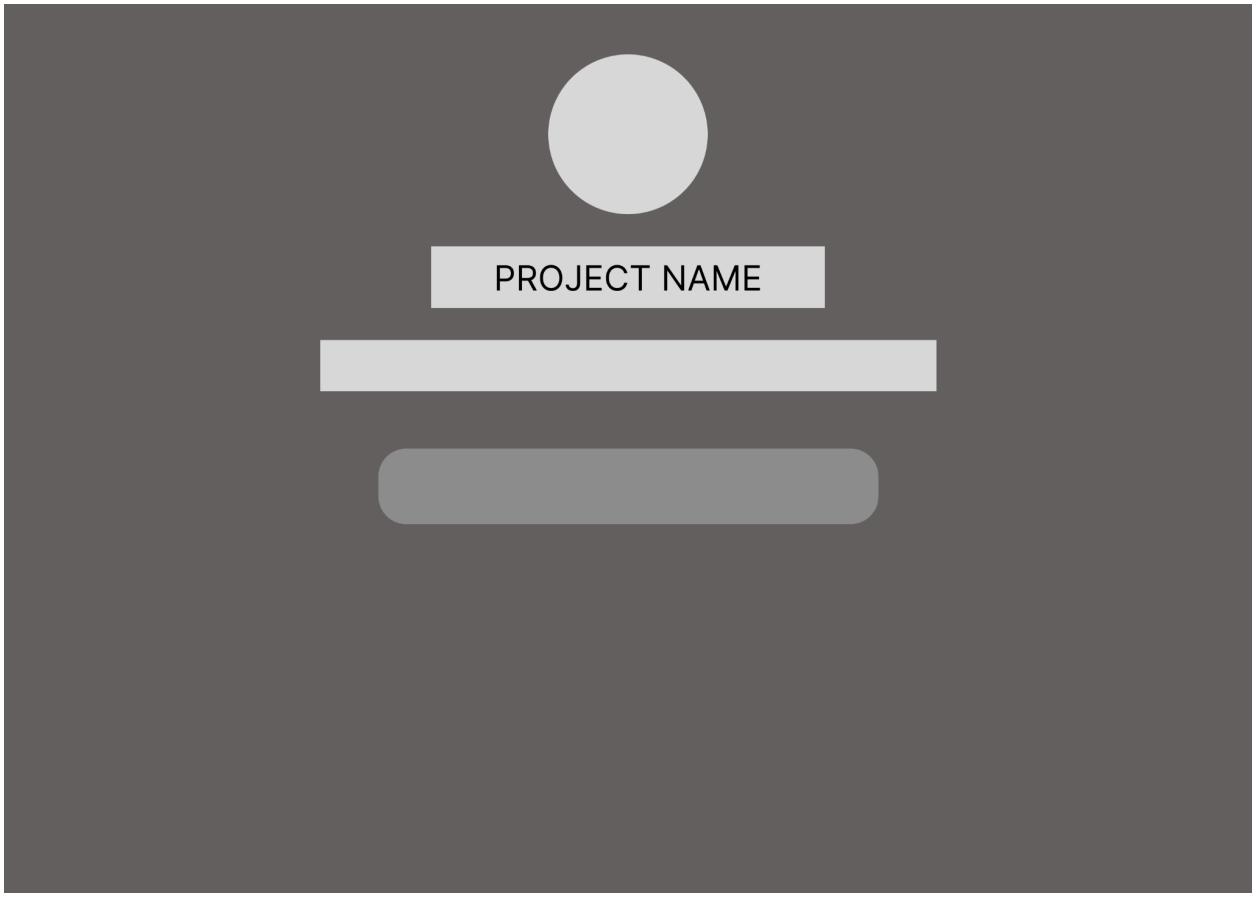
- id (UUID, PRIMARY KEY)
- user_id (UUID, FOREIGN KEY → users.id, NOT NULL)
- token (VARCHAR(500), UNIQUE, NOT NULL)
- expires_at (TIMESTAMP, NOT NULL)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- revoked_at (TIMESTAMP, NULLABLE)

7.0 UI/UX DESIGN

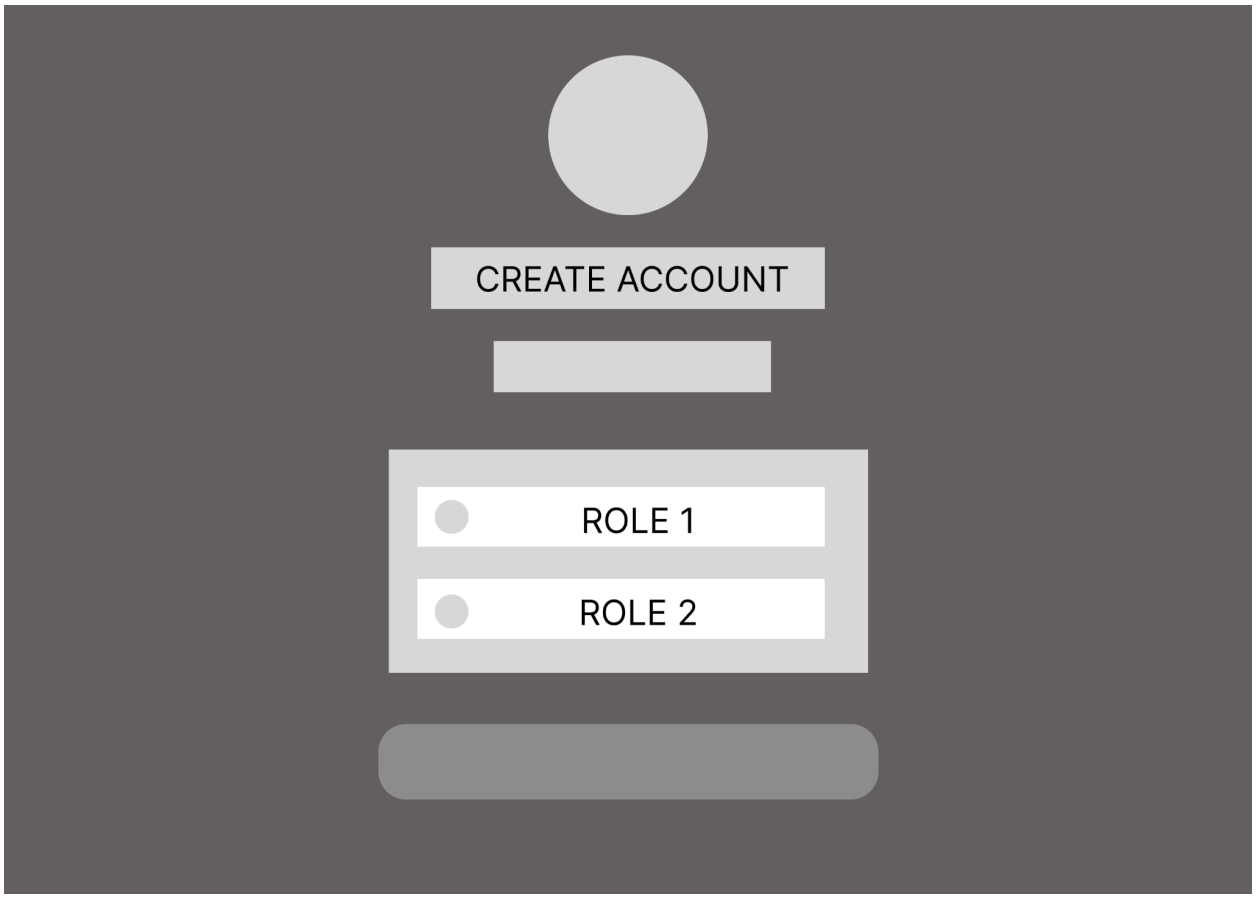
7.1 Web Application Wireframes

Note: This should be wireframes from Figma

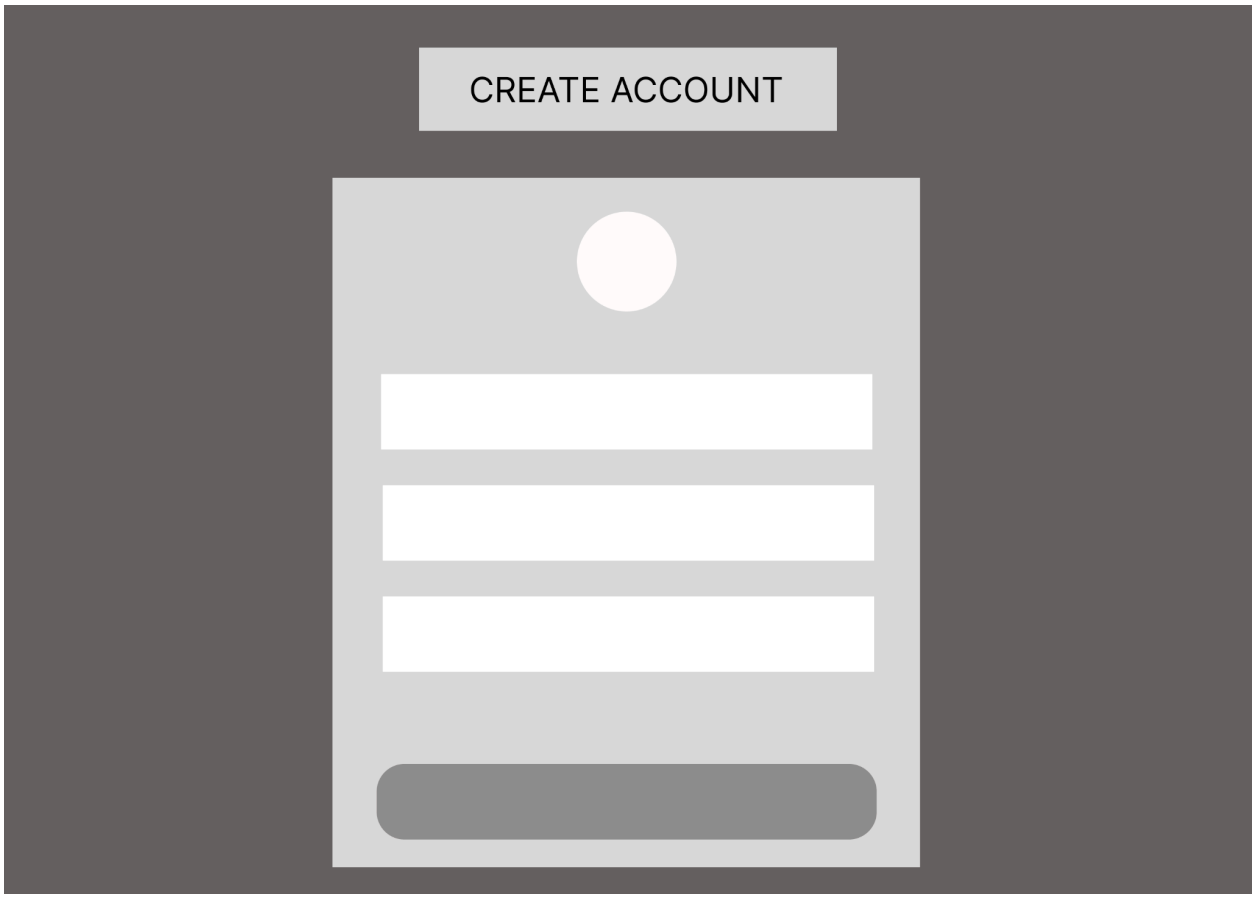
Splash Screen



Register (Role Selection) Page

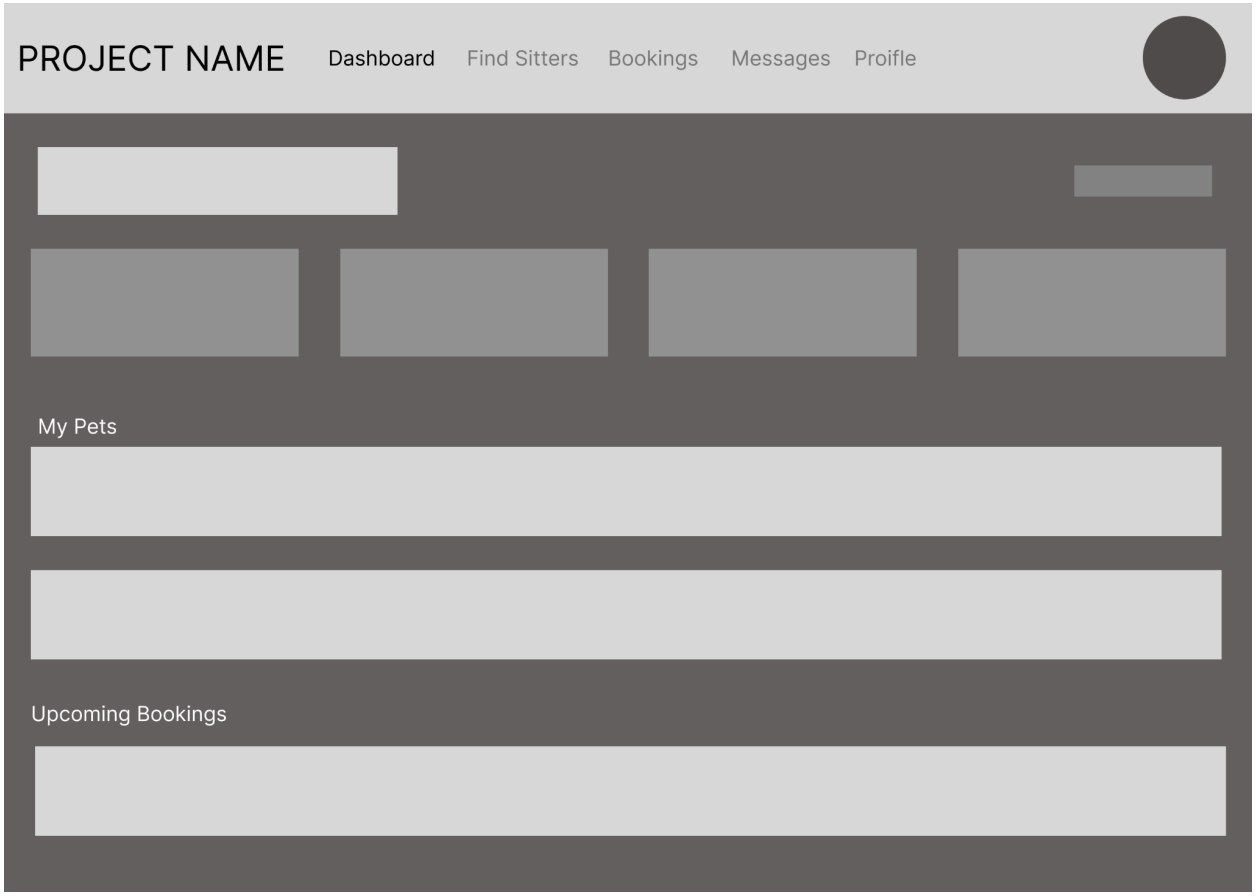


Register Page

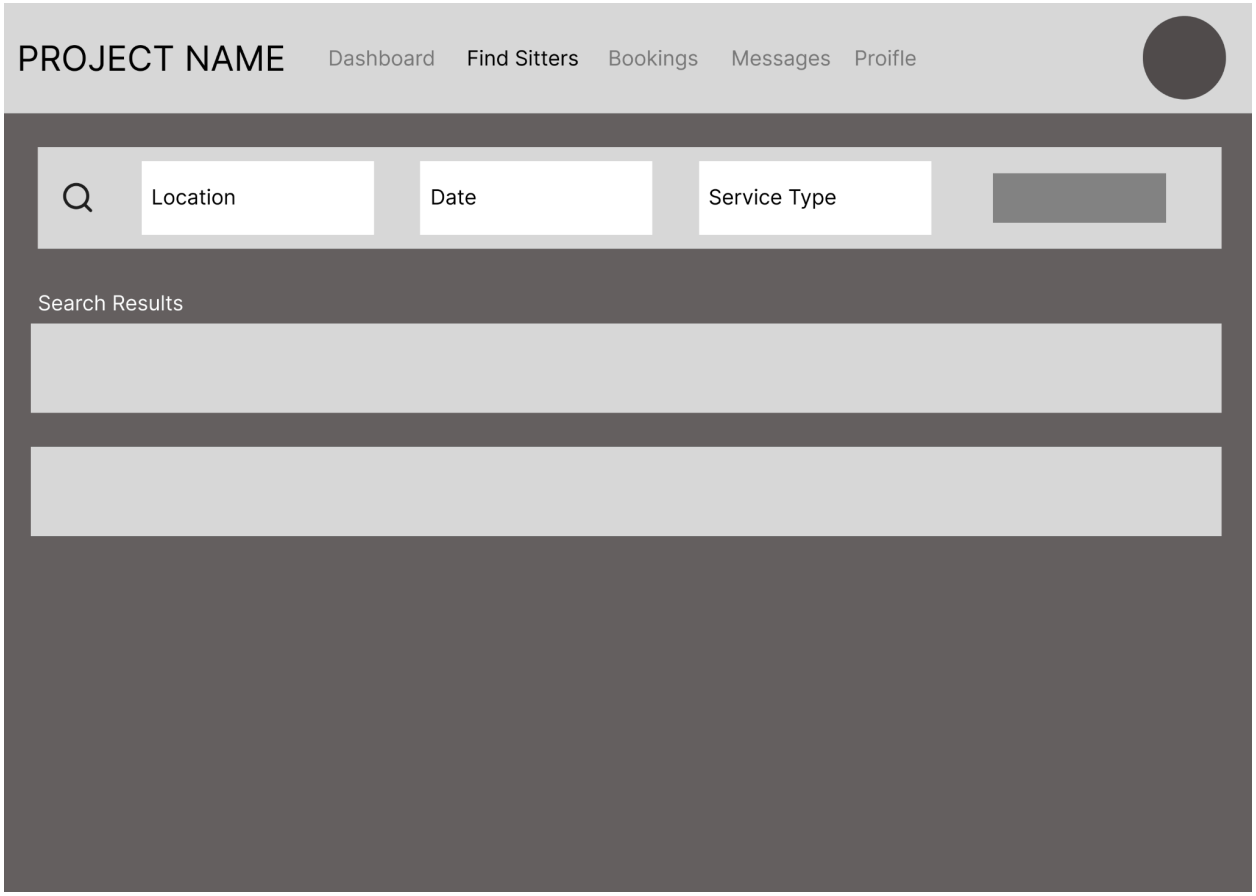


Login Page

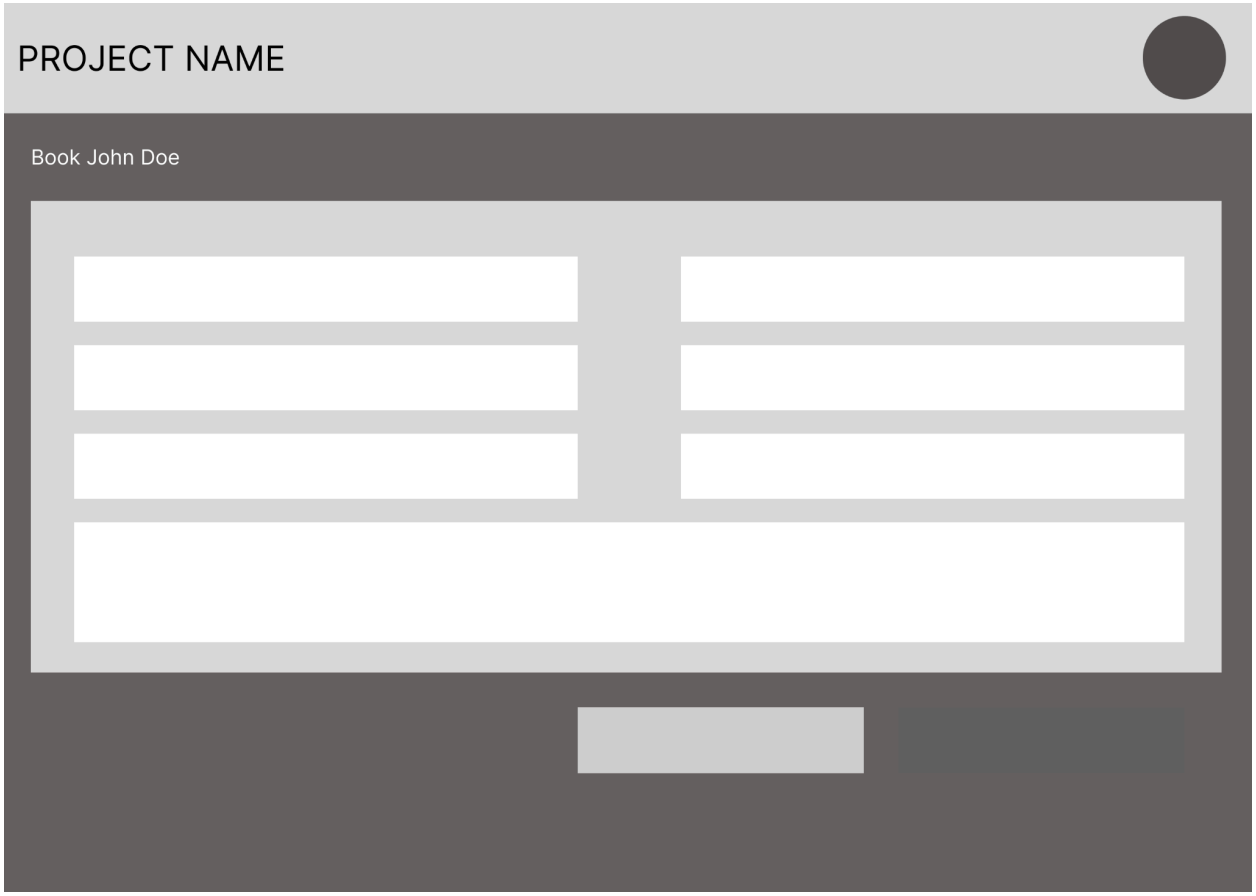
Pet Owner Dashboard Page



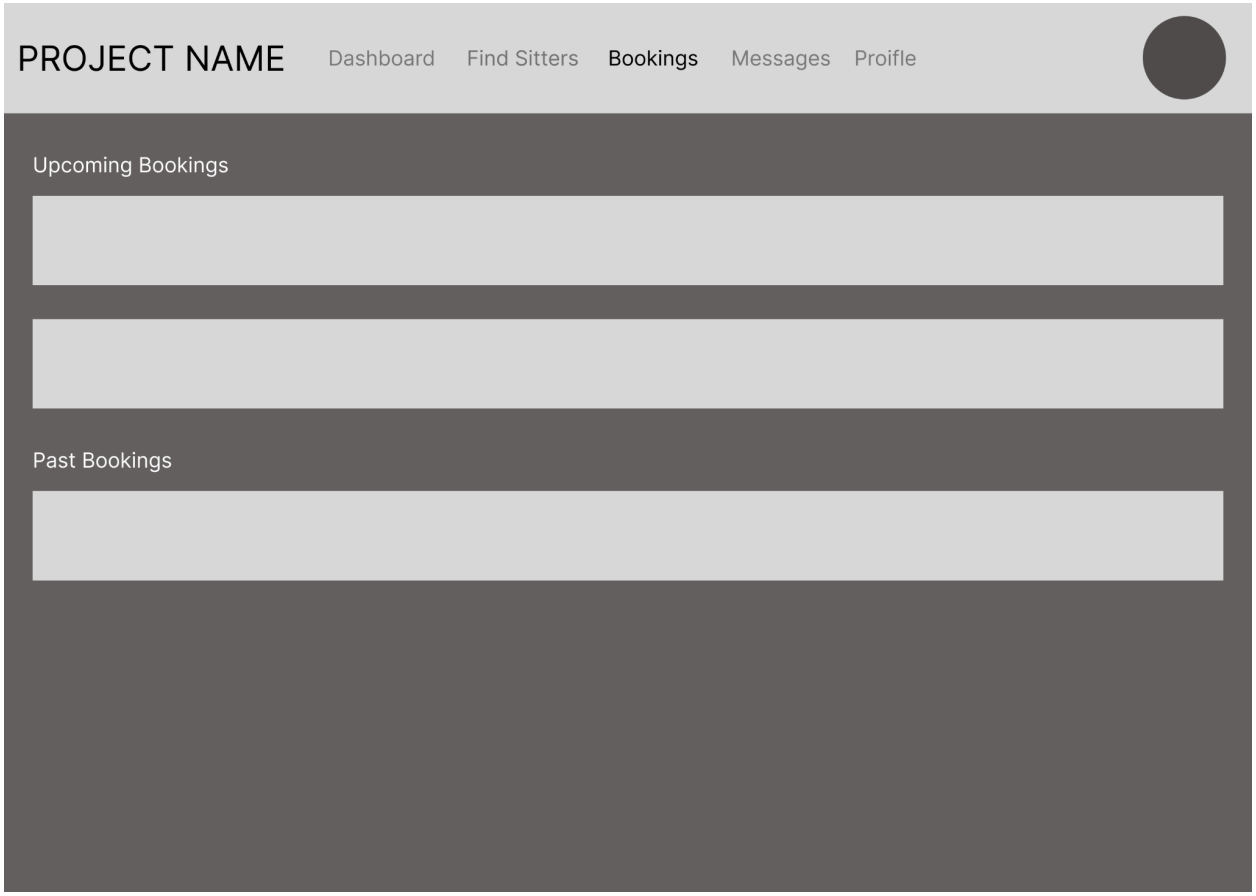
Pet Owner Find Sitter Page



Pet Owner Book Sitter Page

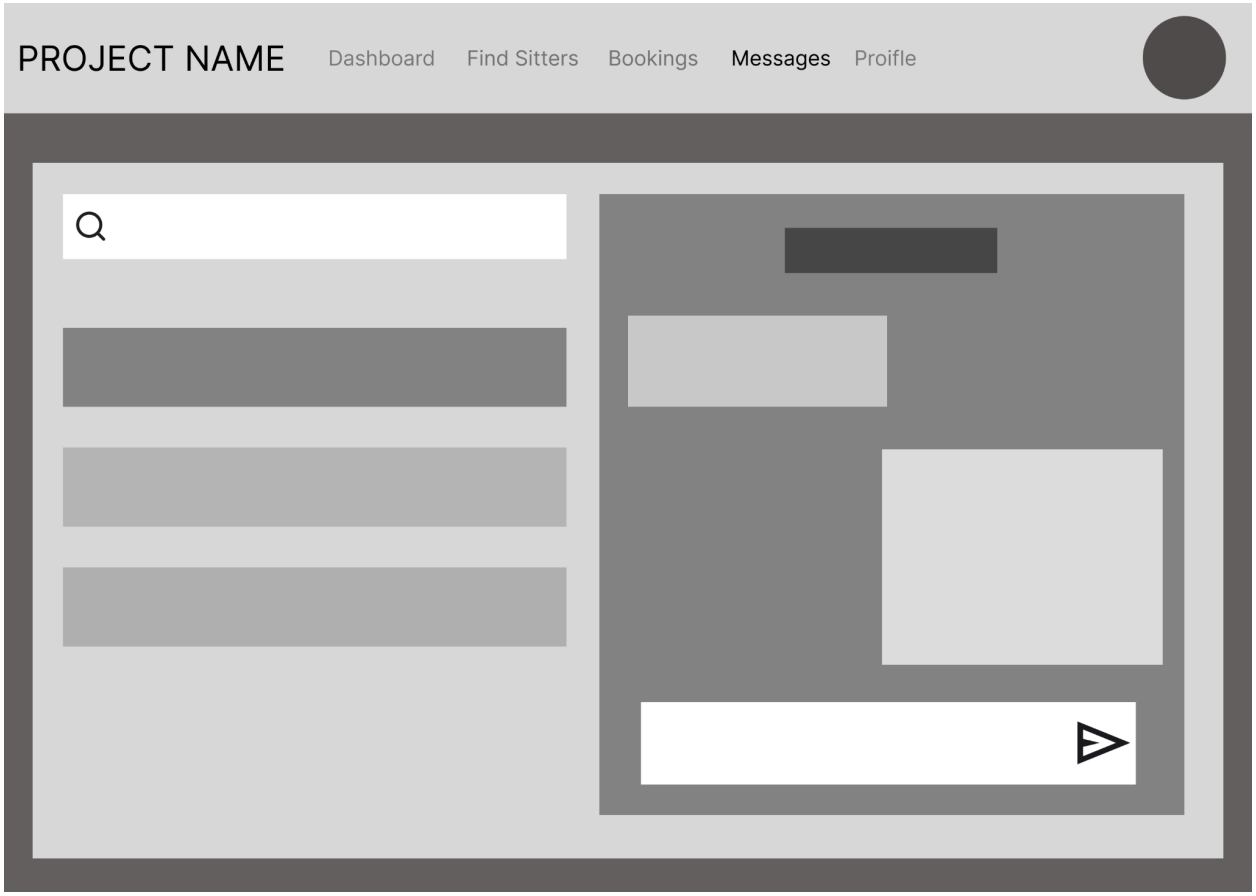


Pet Owner Bookings Page

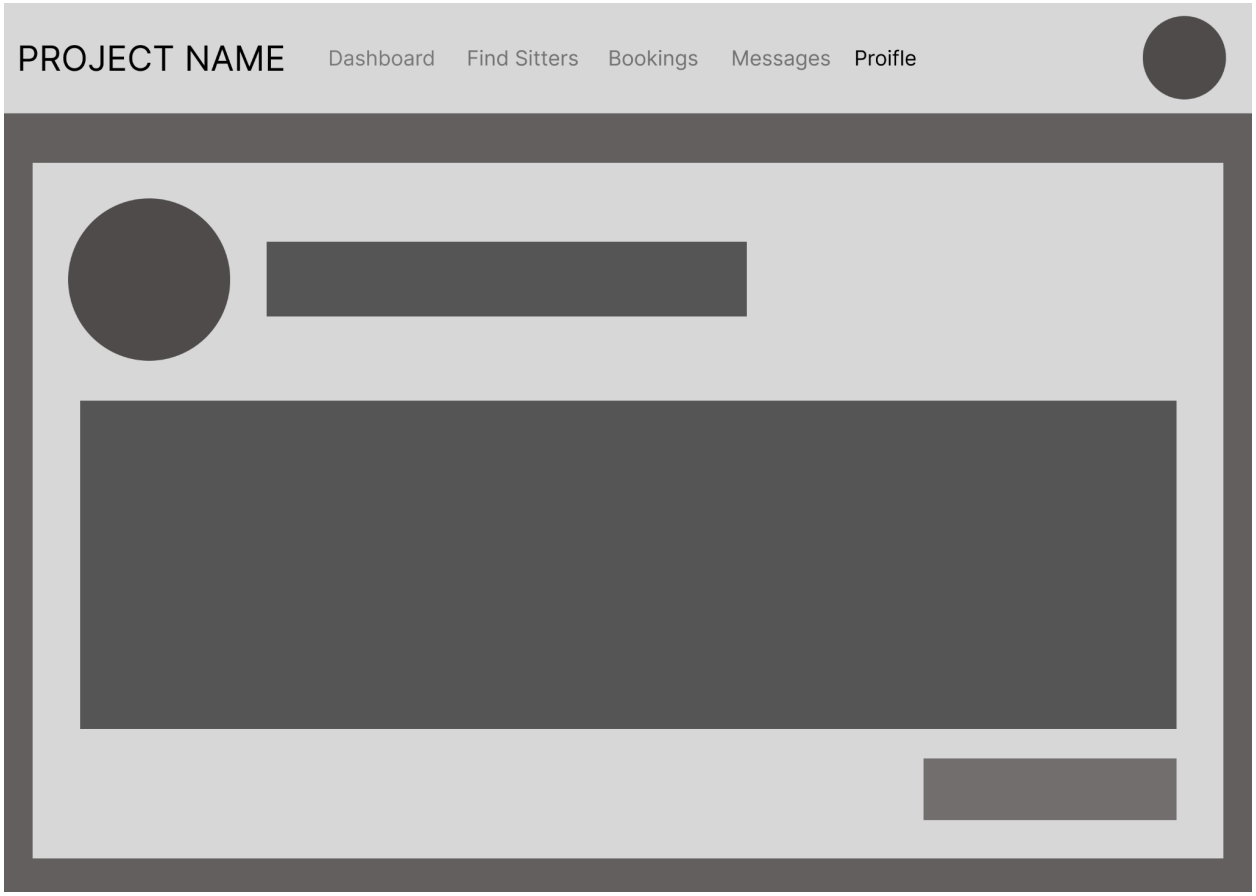


Pet Owner Review Sitter Page

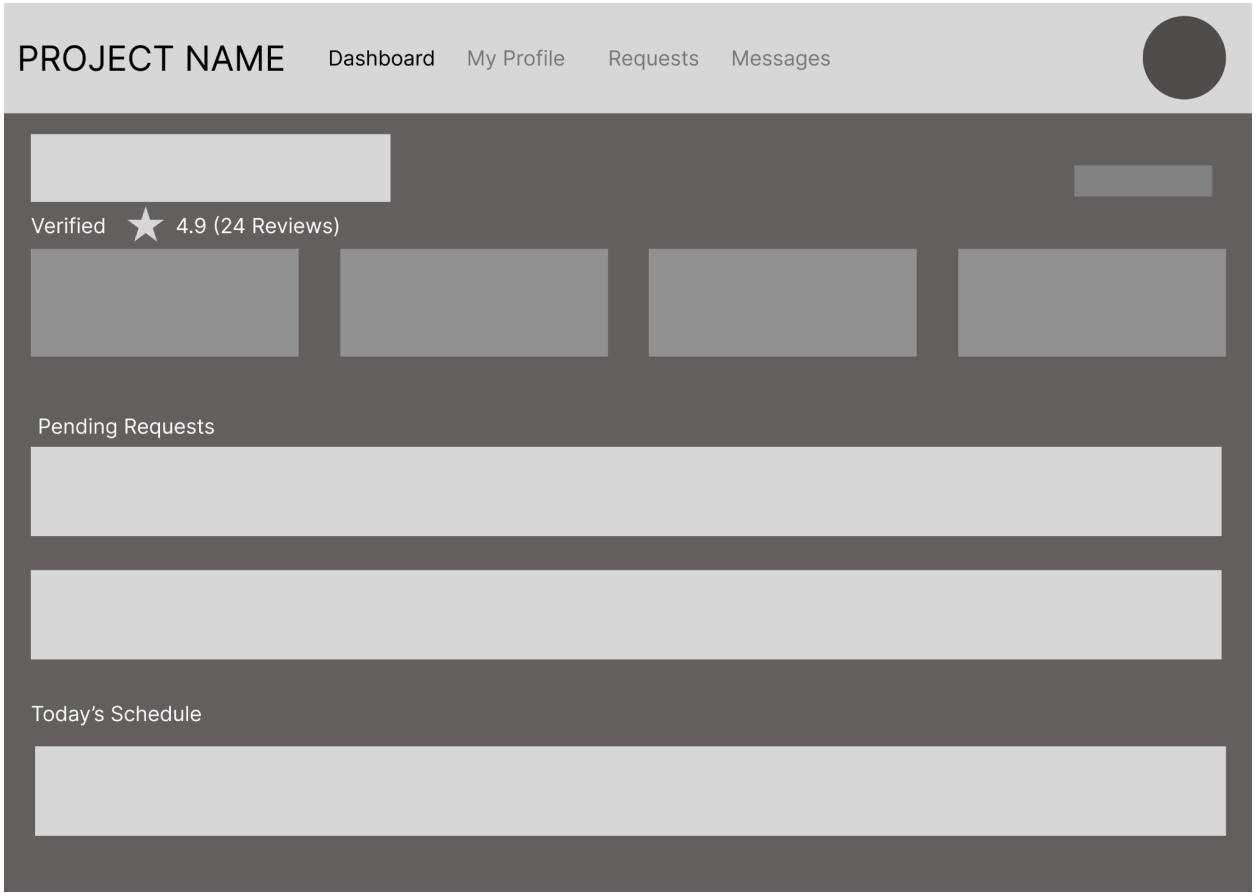
Pet Owner Messages Page



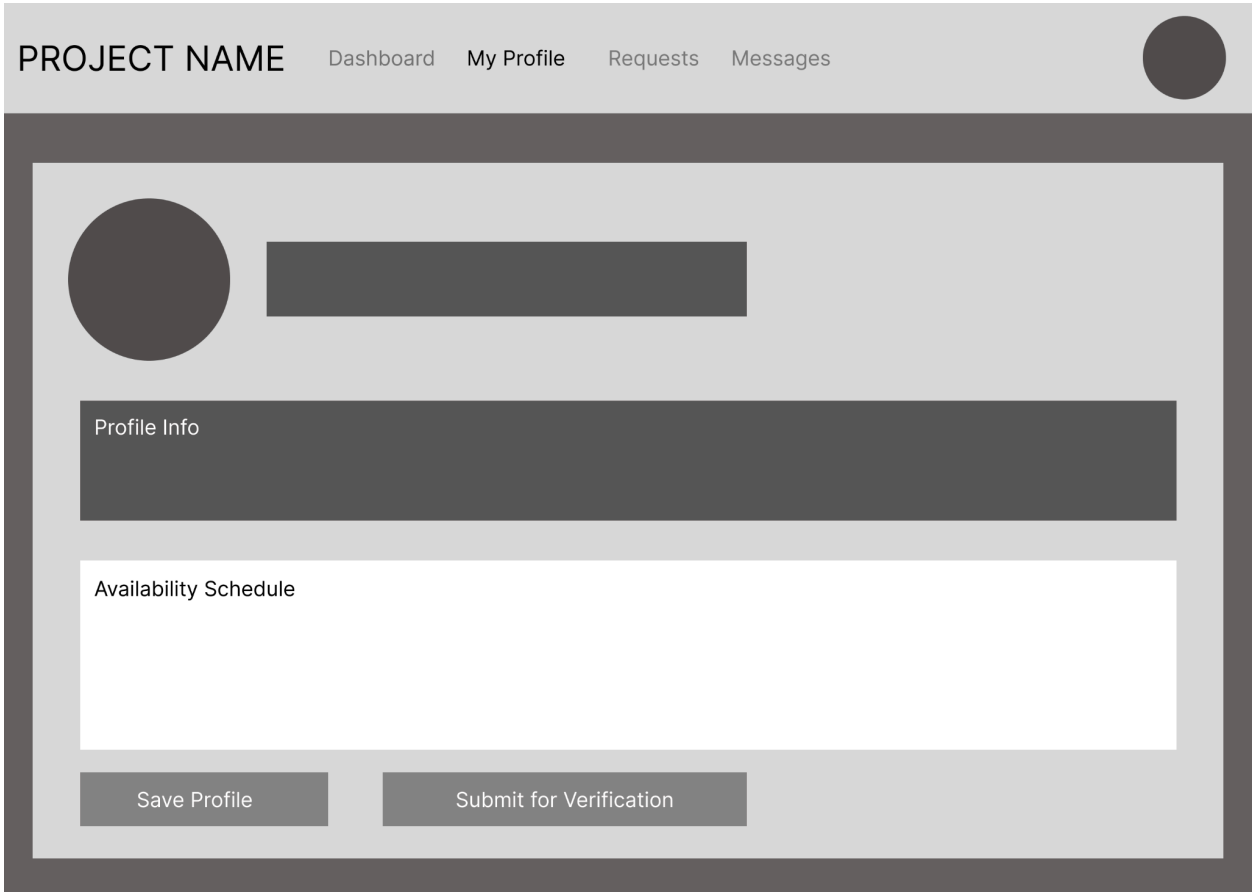
Pet Owner Profile Page



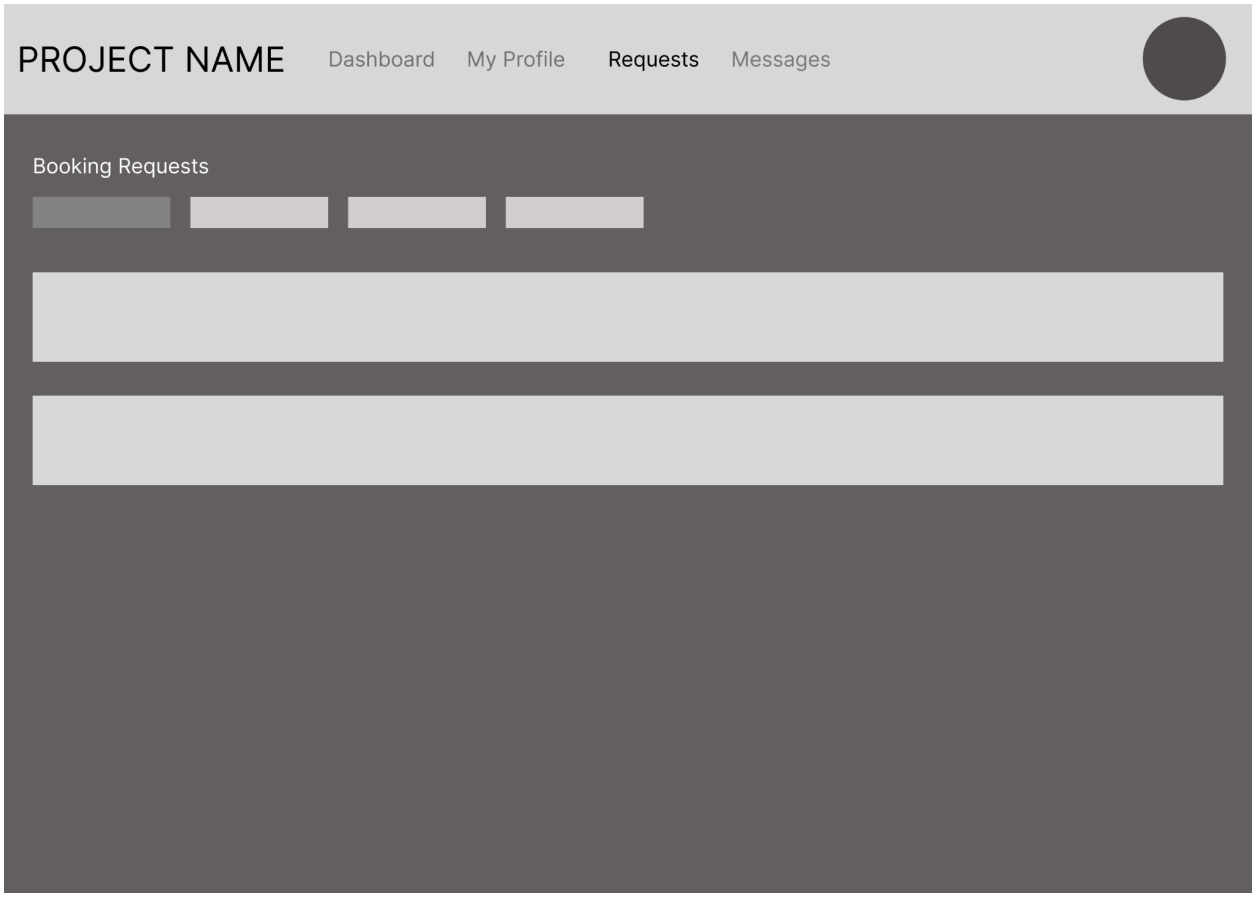
Pet Sitter Dashboard Page



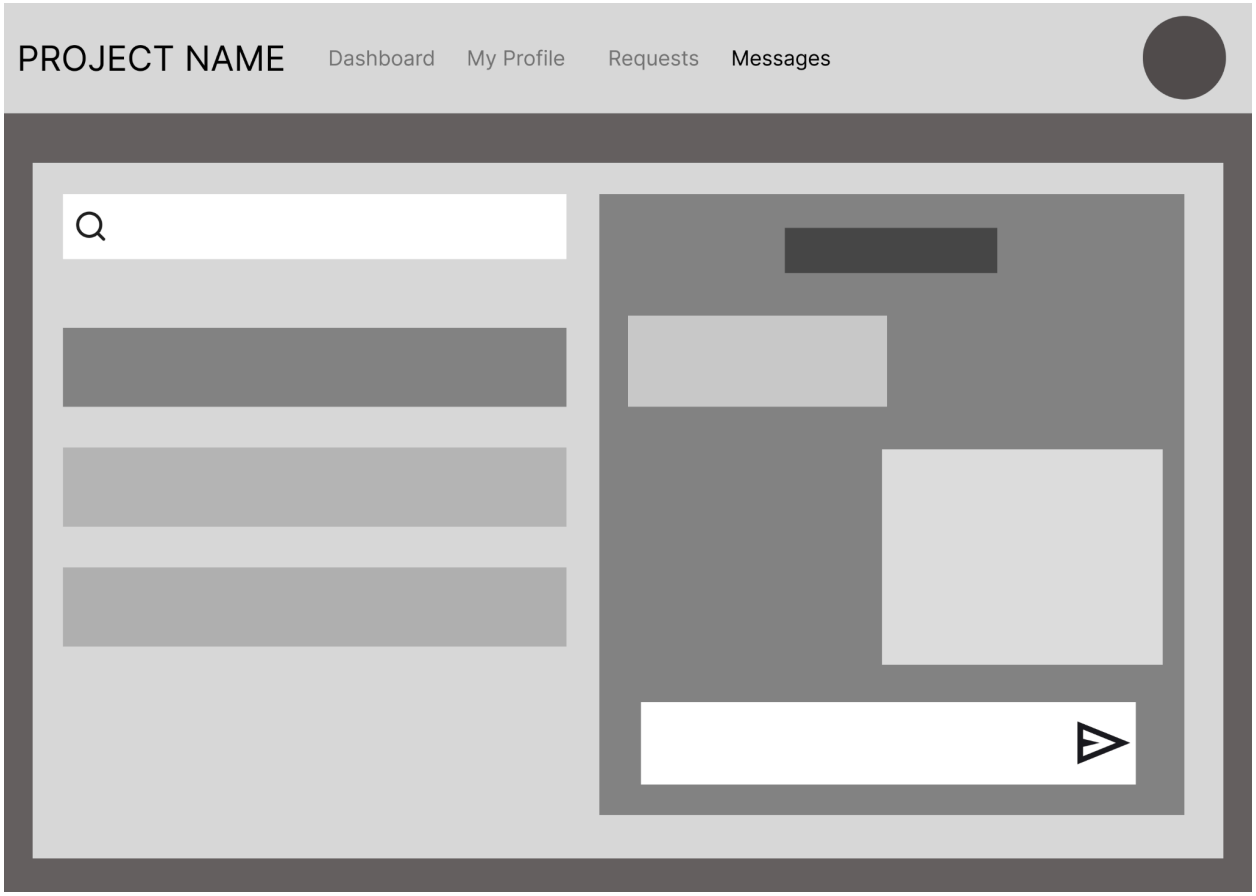
Pet Sitter My Sitter Profile Page



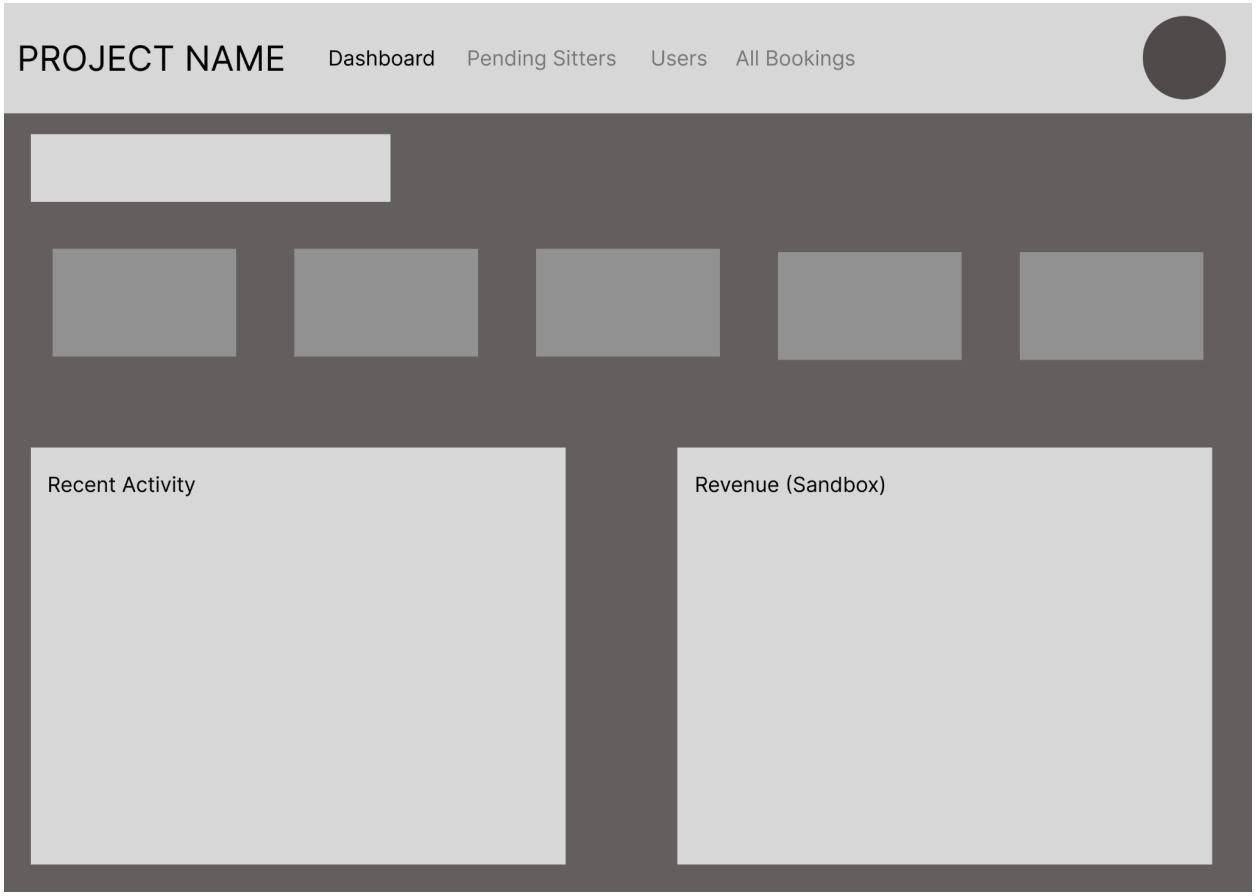
Pet Sitter Requests Page



Pet Sitter Messages Page



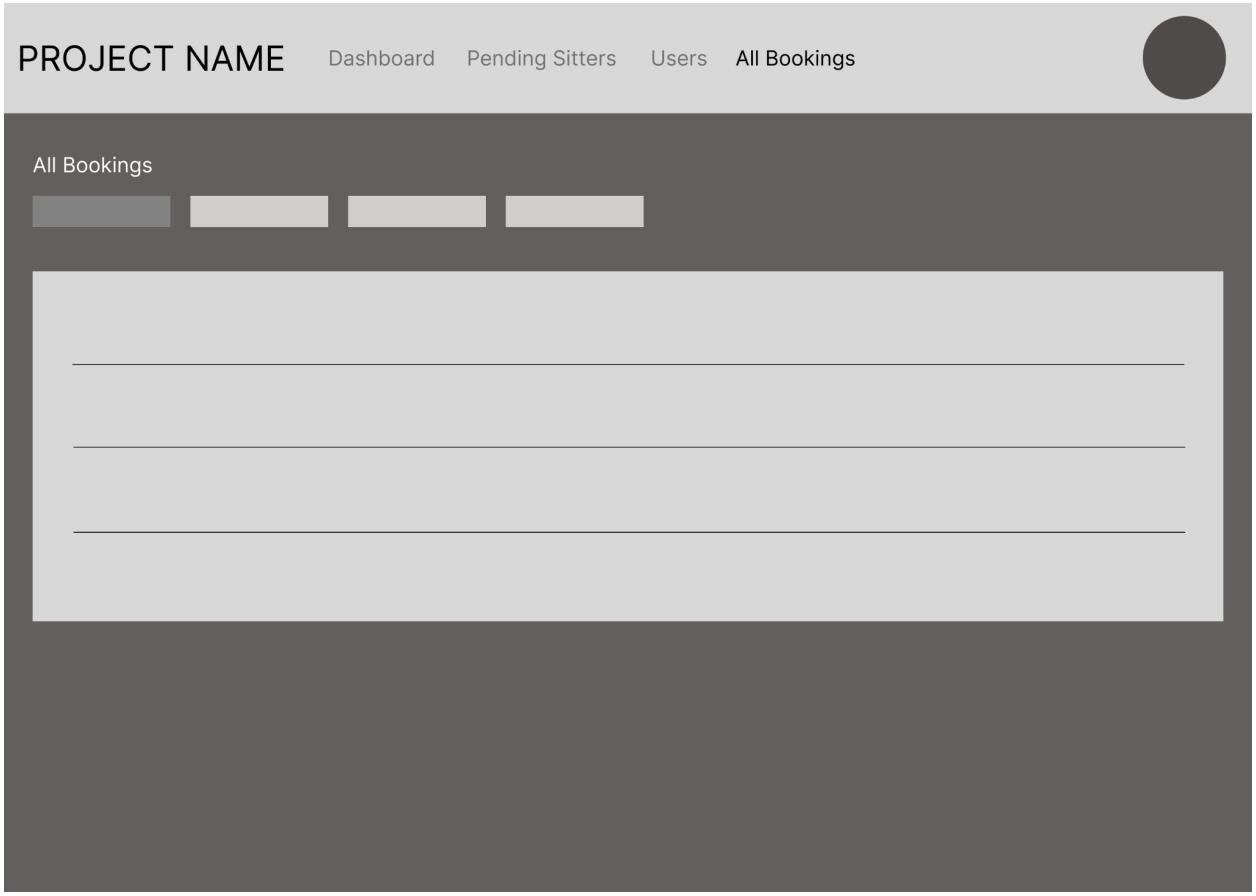
Admin Dashboard Page



Admin Pending Sitters Page

Admin Users Management Page

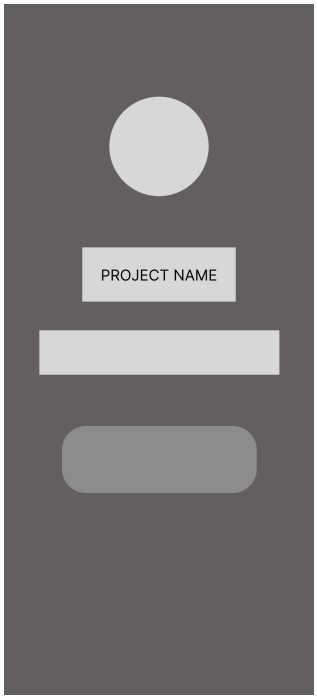
Admin All Bookings Page



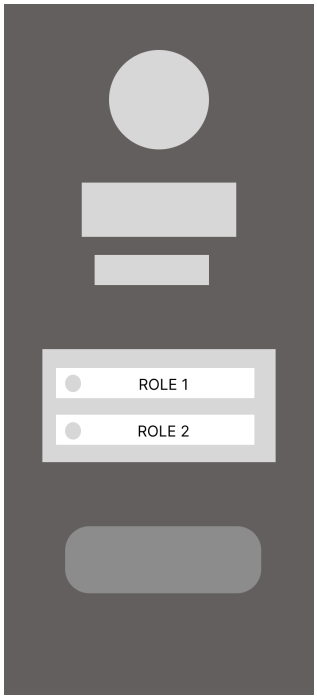
7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

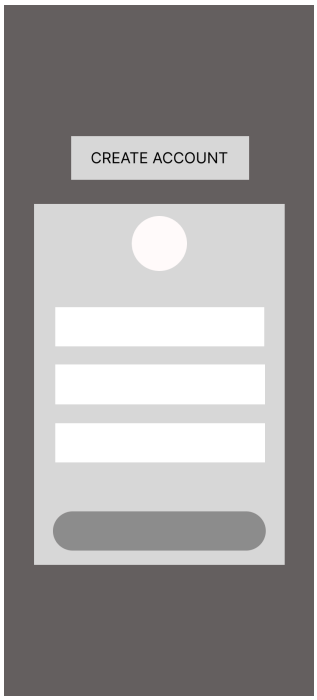
Splash Screen



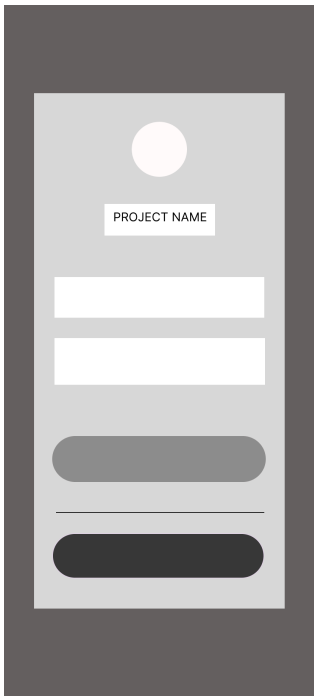
Register (Role Selection) Screen



Registration Screen



Login Screen

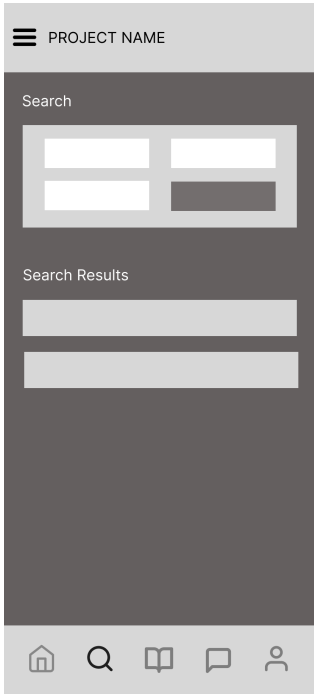


Pet Owner Bottom Navigation

[Dashboard] [Find Sitters] [Bookings] [Messages] [Profile]

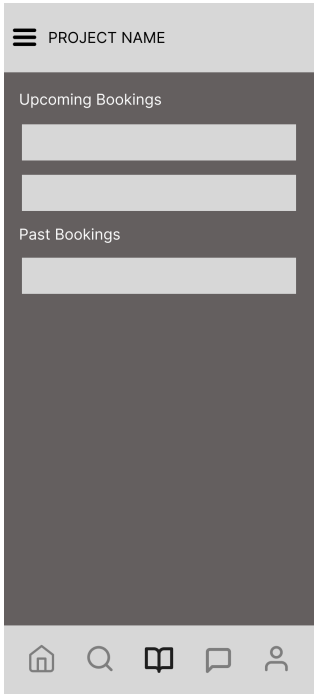
Pet Owner Dashboard Screen

Pet Owner Find Sitters Screen

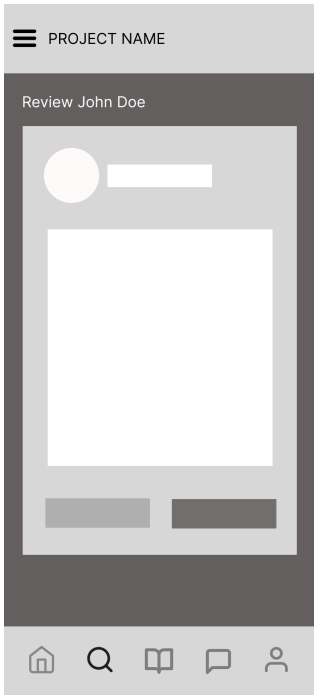


Pet Owner Book Sitter Screen

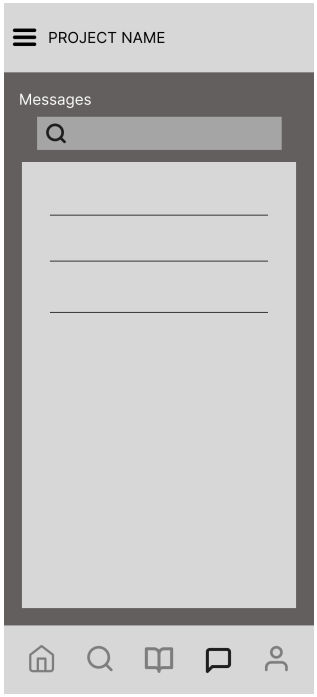
Pet Owner Bookings Screen



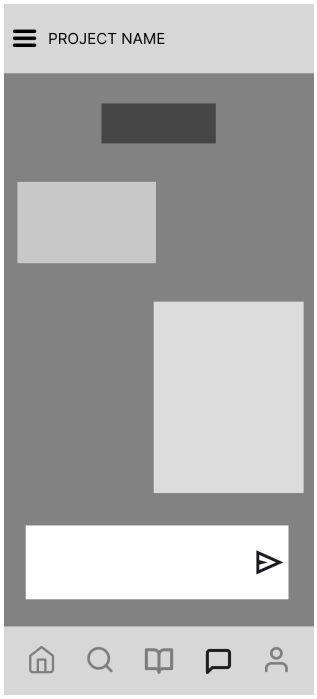
Pet Owner Review Sitter Screen



Pet Owner Messages Screen



Pet Owner Messages Screen

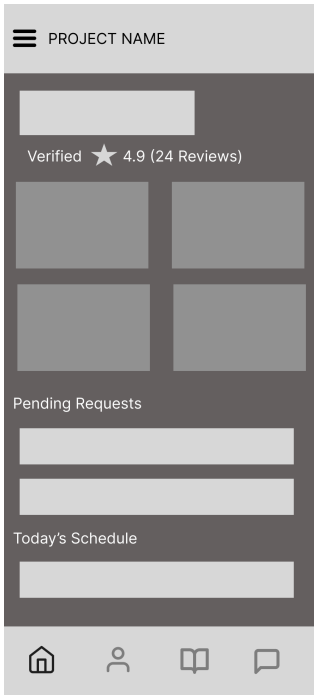


Pet Owner Profile Screen

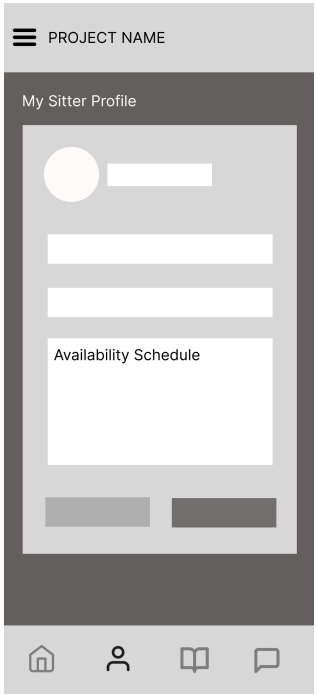
Pet Owner Bottom Navigation

[Dashboard] [My Sitter Profile] [Requests] [Messages]

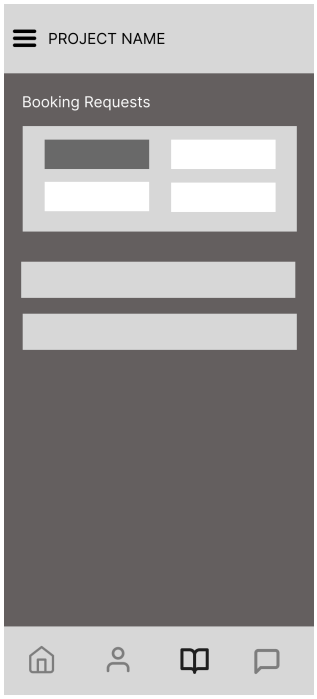
Pet Sitter Dashboard Screen



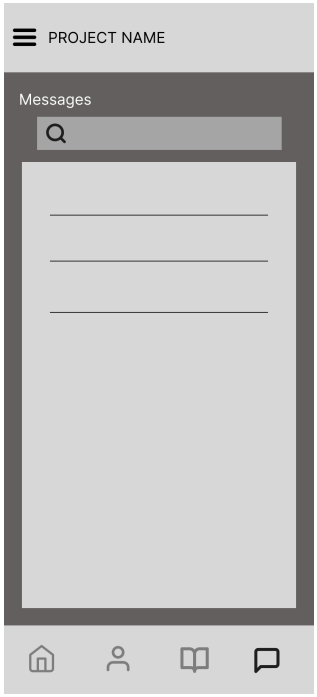
Pet Sitter My Sitter Profile Screen



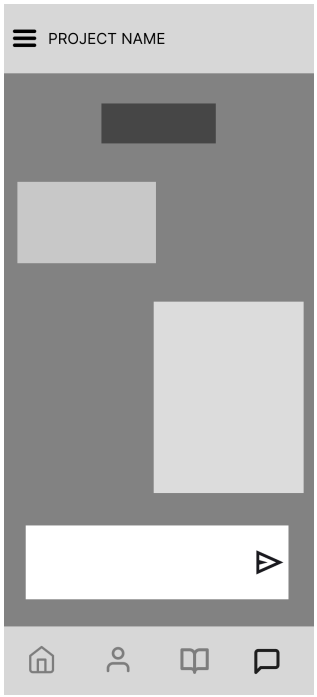
Pet Sitter Requests Screen



Pet Sitter Messages Screen



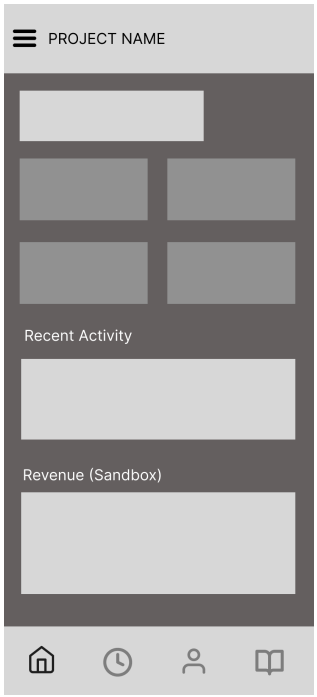
Pet Sitter Messages Screen



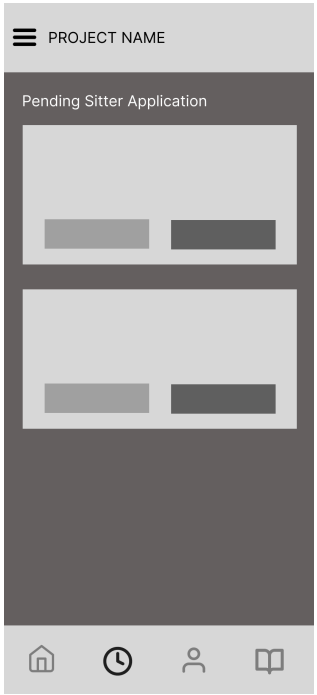
Pet Owner Bottom Navigation

[Dashboard] [Pending Sitters] [User Management] [All Bookings]

Admin Dashboard Screen

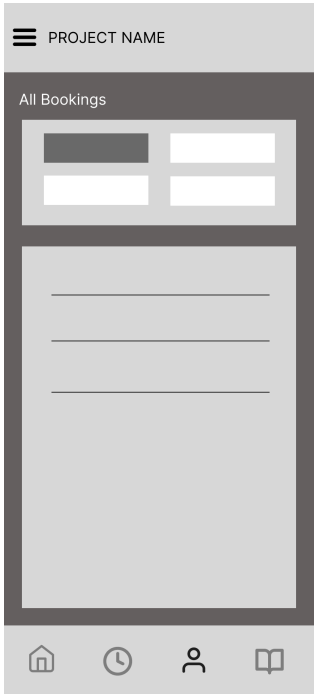


Admin Pending Sitters Screen



Admin User Management Screen

Admin All Bookings Screen



Mobile-Specific Features:

- Touch-optimized buttons sized minimum 44×44 pixels for easy tapping
- Bottom navigation bar on all screens for one-thumb access to main features
- Large input fields with mobile keyboard optimization (email/number keyboards auto-trigger)
- Pull-to-refresh on booking history and sitter search screens
- Hamburger menu (☰) for additional functions
- Simplified forms with progressive disclosure (show only essential fields first)

Design System:

Colors

- Primary (buttons, active states): Soft Peach #FFD8B9
- Secondary (secondary actions): Blush Pink #FFB6C1
- Success (confirmations, verified status): Mint Green #B6E5D8
- Warning (pending states): Butter Yellow #FFF9C4
- Error (errors, cancellations): Coral Pink #FFCCBC
- Neutral Light (background surfaces): Cream White #FFF8F0
- Neutral Medium (placeholder text, borders): Light Gray #D3D3D3
- Neutral Dark (primary text): Charcoal Gray #333333

Typography

- Font Family: Inter (system default for readability)
- Heading 1 (Screen titles): 20px, SemiBold, Charcoal Gray #333333
- Heading 2 (Section headers): 16px, SemiBold, Charcoal Gray #333333
- Body (Primary text): 14px, Regular, Charcoal Gray #333333
- Caption (Helper text): 12px, Regular, Light Gray #D3D3D3
- Line Height: 1.5× font size for readability

Spacing

- Base unit: 8px grid system
- Small gap: 8px (between related items like pet cards)
- Medium gap: 16px (between sections like "My Pets" and "Quick Actions")
- Large gap: 24px (between major screen areas like header and content)
- Edge padding: 16px (horizontal screen margins on mobile)

Components

- Buttons: Rounded corners (12px), soft peach fill (#FFD8B9), charcoal text (#333333), minimum 44×44px touch target
- Input Fields: 48px height, 2px border (Light Gray #D3D3D3), rounded corners (10px), cream white background (#FFF8F0)
- Cards: Cream white background (#FFF8F0), subtle shadow (0px 2px 6px rgba(0,0,0,0.05)), rounded corners (16px), soft borders
- Badges: Small rounded tags with pastel fills (e.g., "CONFIRMED" in mint green, "PENDING" in butter yellow)
- Icons: 24×24px line icons in charcoal gray (#333333) with consistent stroke width (2px)

Responsive Breakpoints

- Mobile: 360px–767px (single column layout, bottom navigation)
- Desktop: 1024px+ (3+ column layouts, persistent sidebar for admin views)

8.0 PLAN

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- Day 1-2: Project setup and documentation
- Day 3-4: Complete FRS and NFR
- Day 5-7: System architecture design

Week 2: Detailed Design

- Day 1-2: API specification
- Day 3-4: Database design
- Day 5-6: UI/UX wireframes
- Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-5)

Week 3: Foundation

- Day 1: Spring Boot setup with dependencies (Spring Security, JPA, JWT)
- Day 2: Database configuration and entity classes

- Day 3: JWT authentication implementation
- Day 4: User registration and login endpoints
- Day 5: Role-based access control (PET_OWNER, PET_SITTER, ADMIN)

Week 4: Core Features

- Day 1: Pet profile CRUD operations
- Day 2: Sitter profile management endpoints
- Day 3: Availability calendar and scheduling
- Day 4: Booking system (create, view, cancel)
- Day 5: Review and rating system

Week 5: Advanced Features

- Day 1: Sandbox payment simulation (no real transactions)
- Day 2: Admin endpoints (sitter verification, user management)
- Day 3: Search and filtering for sitters
- Day 4: Error handling and validation
- Day 5: API documentation and testing

Phase 3: Web Application (Week 6-7)

Week 6: Frontend Foundation

- Day 1: React setup with TypeScript
- Day 2: Authentication pages (Login, Register)
- Day 3: Pet Owner dashboard and pet profile management
- Day 4: Sitter search and profile viewing
- Day 5: Booking flow implementation

Week 7: Complete Web Features

- Day 1: Booking history and status tracking
- Day 2: Review/rating submission
- Day 3: Admin dashboard (pending verifications, user management)
- Day 4: Responsive design polish (mobile, tablet, desktop)
- Day 5: API integration and testing

Phase 4: Mobile Application (Week 8-9)

Week 8: Android Foundation

- Day 1: Android Studio setup and project structure
- Day 2: Authentication screens (Login, Register)
- Day 3: Pet profile management
- Day 4: Pet Owner dashboard
- Day 5: Sitter search and booking flow

Week 9: Complete Mobile App

- Day 1: Booking history and status
- Day 2: Review/rating submission
- Day 3: UI polish and testing on emulator/device
- Day 4: Bug fixes and optimization
- Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 10)

Week 10: Final Integration

- Day 1: End-to-end testing across web and mobile platforms
- Day 2: Bug fixes and performance optimization
- Day 3: Security review
- Day 4: Documentation updates
- Day 5: Deployment and project submission

Milestones

- M1 (End Week 2): All design documents complete (SDD, ERD, wireframes)
- M2 (End Week 5): Backend API fully functional with all endpoints
- M3 (End Week 7): Web application complete and tested
- M4 (End Week 9): Mobile application complete and tested
- M5 (End Week 10): Full system deployed and integrated

Critical Path

- Authentication system (Week 3)

- Database schema and entities (Week 3)
- Pet and sitter profiles (Week 4)
- Booking system (Week 4)
- Web frontend integration (Week 6-7)
- Mobile app development (Week 8-9)
- Cross-platform testing (Week 10)

Risk Mitigation

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Use sandbox mode for payment simulation (no real transactions)